

中国科学技术大学
University of Science and Technology of China

Onix
大规模生产网络的分布式控制平台

USENIX OSDI 2010

授课教师：赵功名
中国科大计算机学院
2026年秋 • 高级计算机网络

Outline

I. 问题与贡献
II. 架构与 NIB
III. 扩展与可靠性
IV. 状态分发与一致性
V. 实现与应用
VI. 实验评估
VII. 小结

背景：OpenFlow 与控制器

OpenFlow：用协议编程交换机

OpenFlow 规定控制器怎样读取和修改交换机的转发行为

交换机按流表处理报文

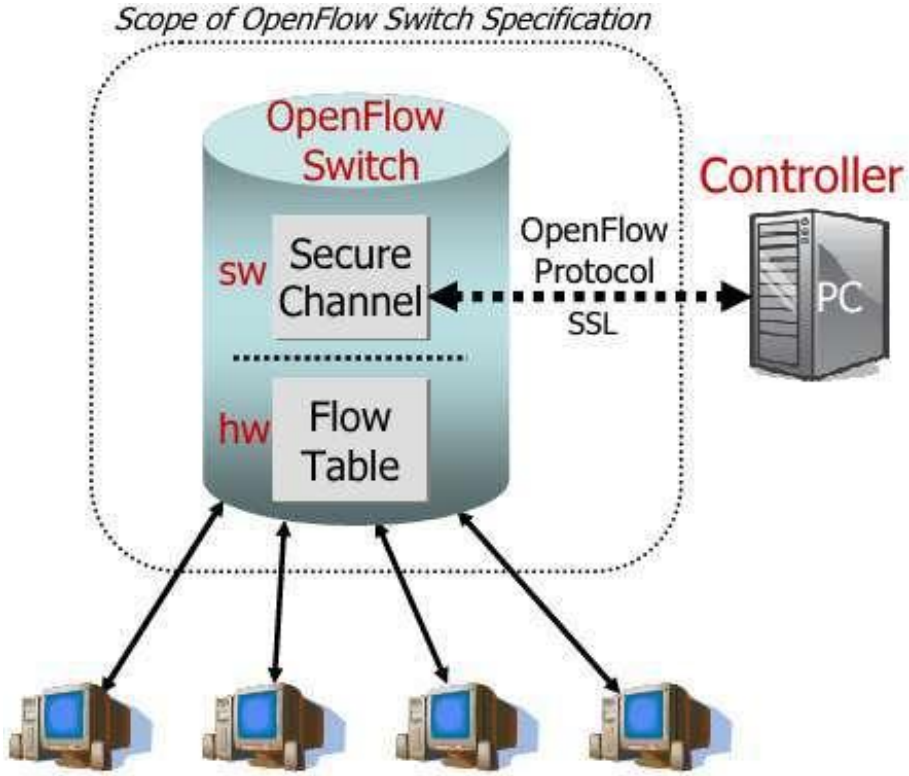
交换机保存流表，按匹配条件执行转发动作

OpenFlow 传递控制信息

控制器通过 OpenFlow 下发规则、查询统计和接收事件

控制应用决定转发策略

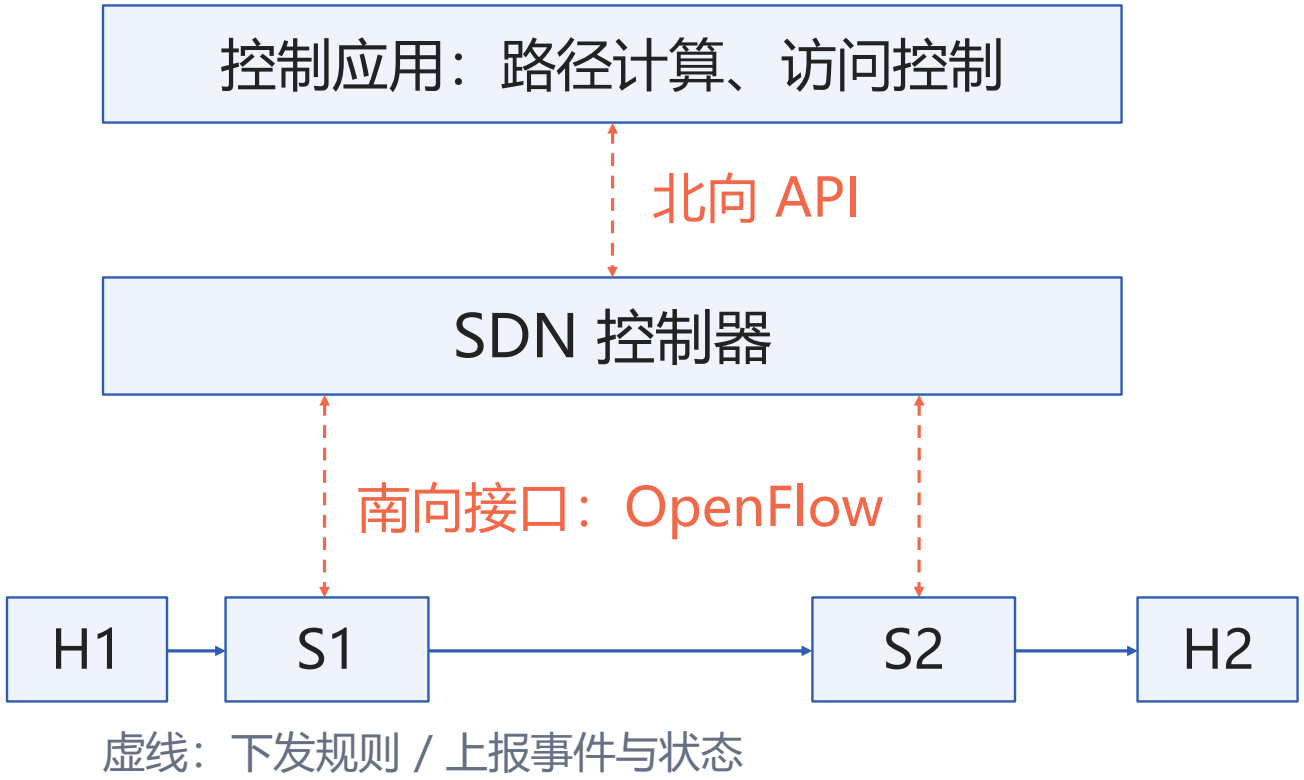
选哪条路径、允许谁通信，由控制应用决定



OpenFlow 规定设备控制方式；下面看它在 SDN 架构中的位置

SDN 架构：OpenFlow 连接控制器与交换机

OpenFlow 连接控制器与交换机：软件作出控制决策，交换机持续转发报文



控制平面

应用结合网络状态与策略计算规则；
平台连接设备并处理事件

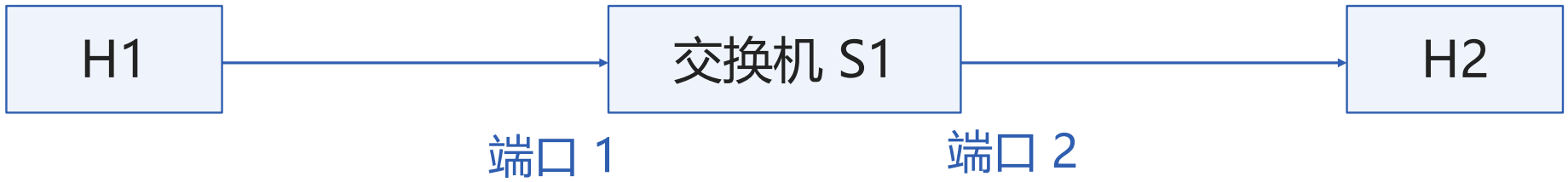
数据平面

交换机按已有规则匹配、转发报文；
下方实线表示业务路径

北向、南向表示接口位置，通信均可双向；交换机具体按流表执行规则

OpenFlow 流表：匹配、动作与统计

交换机执行的规则以流表项保存，包含匹配条件、动作和统计



简化流表示例

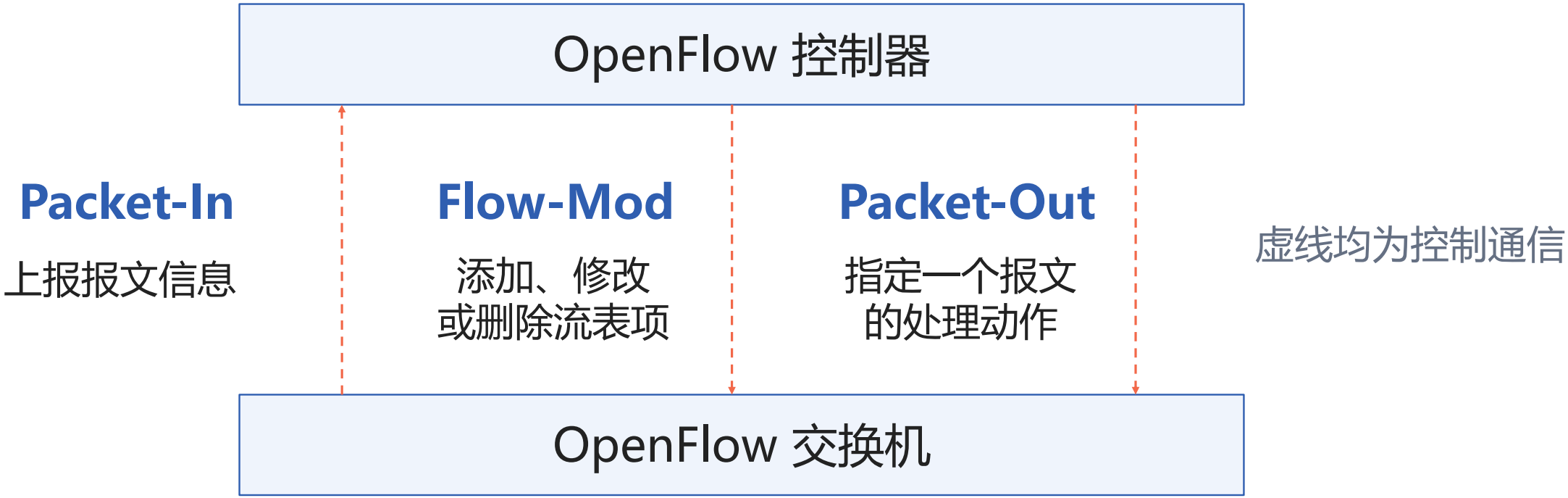
匹配条件	动作	统计
目的 MAC = H2 的 MAC	从端口 2 输出	包数、字节数

匹配字段可以包含入口端口、MAC/IP 地址等；一条规则可匹配多个报文

匹配成功时直接转发；控制器通过控制消息处理上报并修改规则

OpenFlow：三种常见控制消息

围绕流表控制，三种消息分别负责上报报文、修改规则和处理指定报文

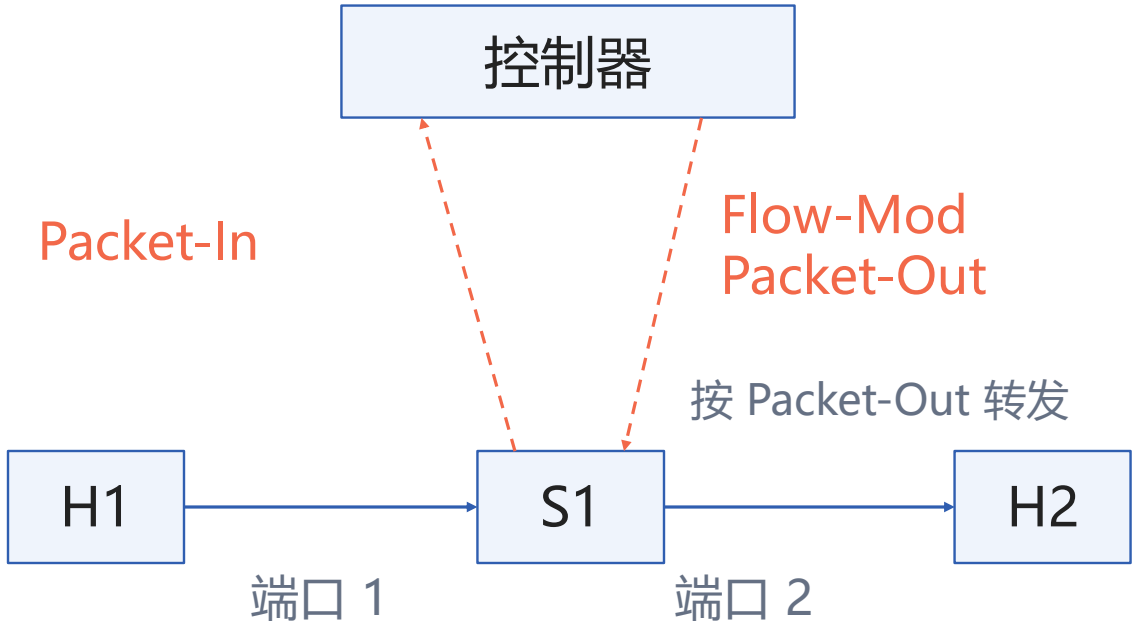


Flow-Mod 改规则；Packet-Out 处理一个指定报文

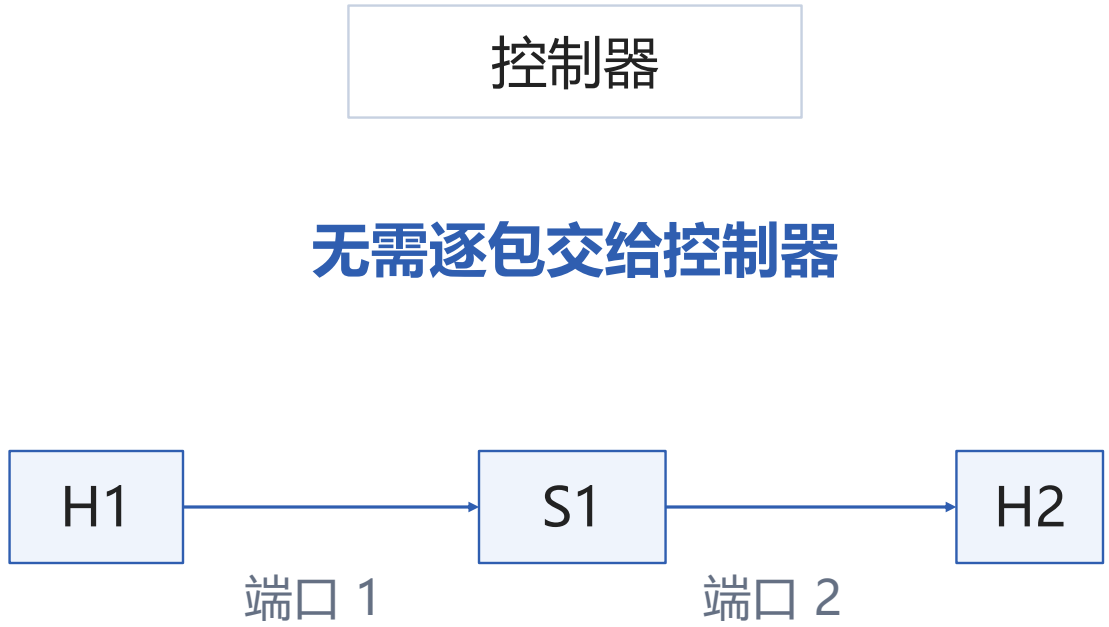
OpenFlow：首个报文和后续报文

把三种消息放进一次转发过程：已知 H2 在端口 2，未匹配报文配置为上送控制器

首次未命中：请求控制处理



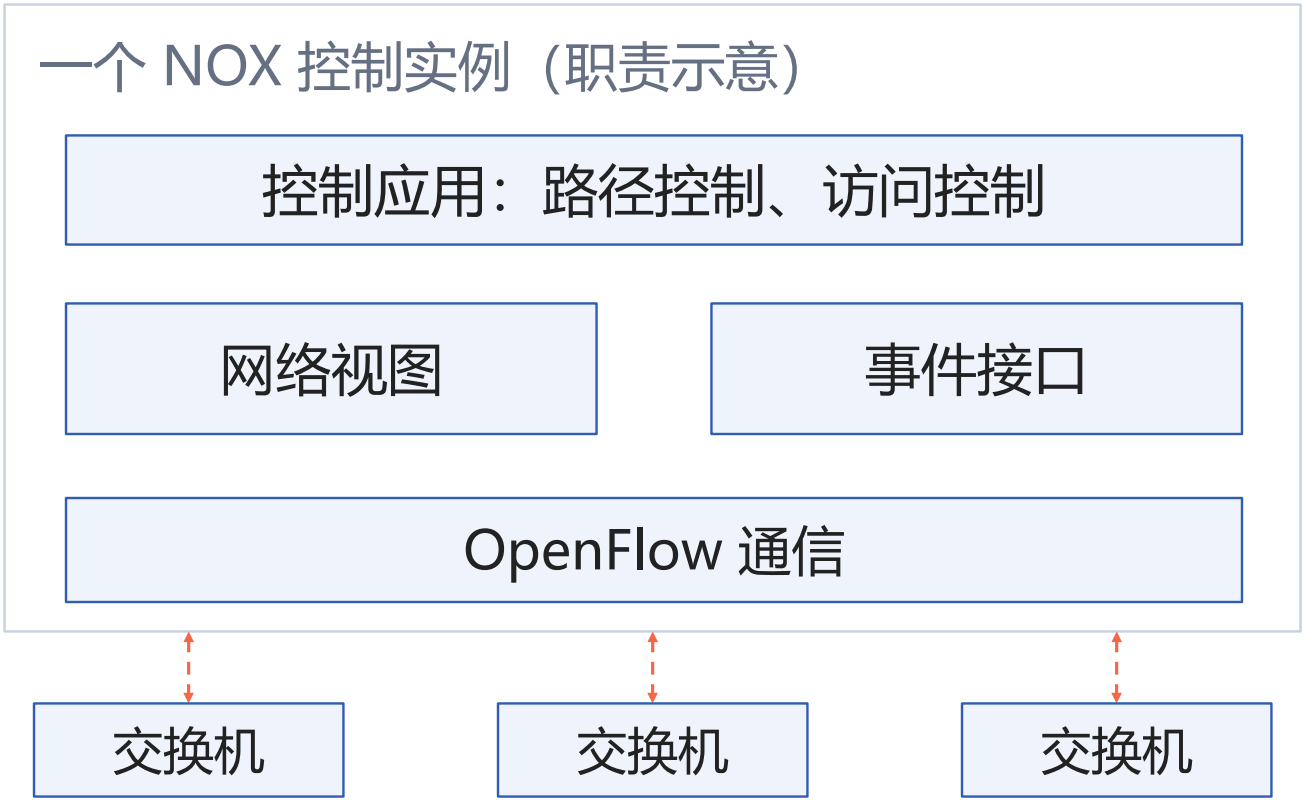
规则安装后：交换机直接转发



规则安装后，后续匹配报文由交换机直接处理

NOX：让控制应用面向整个网络编程

控制器要作出路径决策，还需要网络视图和事件接口；NOX 提供这些平台能力



网络级的编程方式

应用结合拓扑、主机位置等信息，通过事件接口计算路径或访问规则

平台与应用各有职责

NOX 提供设备接入和运行支持；应用决定网络应当怎样工作

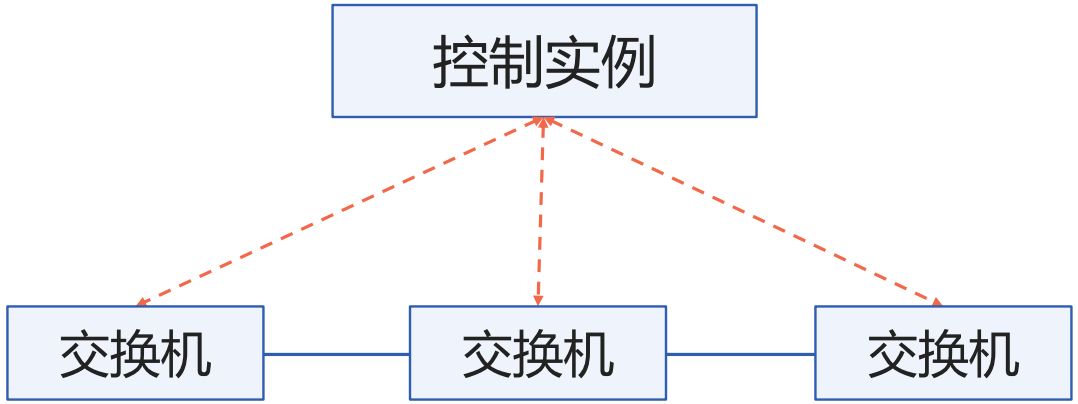
本页画法是单实例，NOX 原设计已讨论多进程

OpenFlow 说明怎样控制设备，NOX 帮助应用使用整个网络的信息

单实例控制器：规模和故障带来的问题

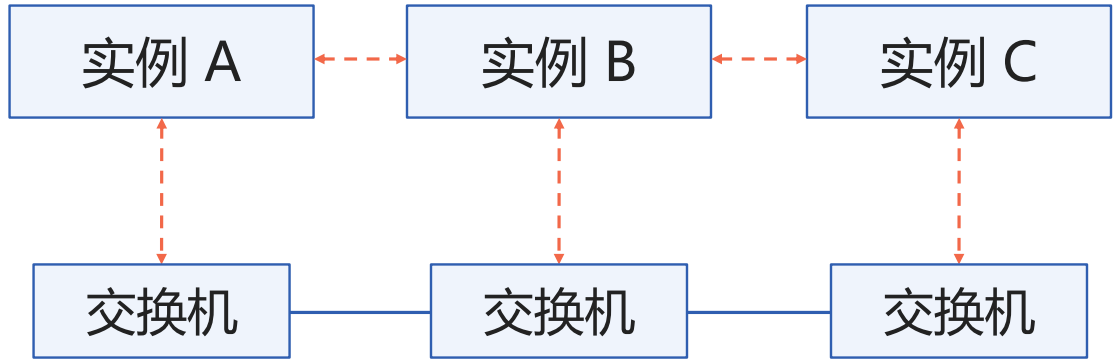
网络扩大后，单实例的状态和工作量可能过大；增加实例又需要解决分工与协作

单实例部署



状态与工作集中

多实例控制平台



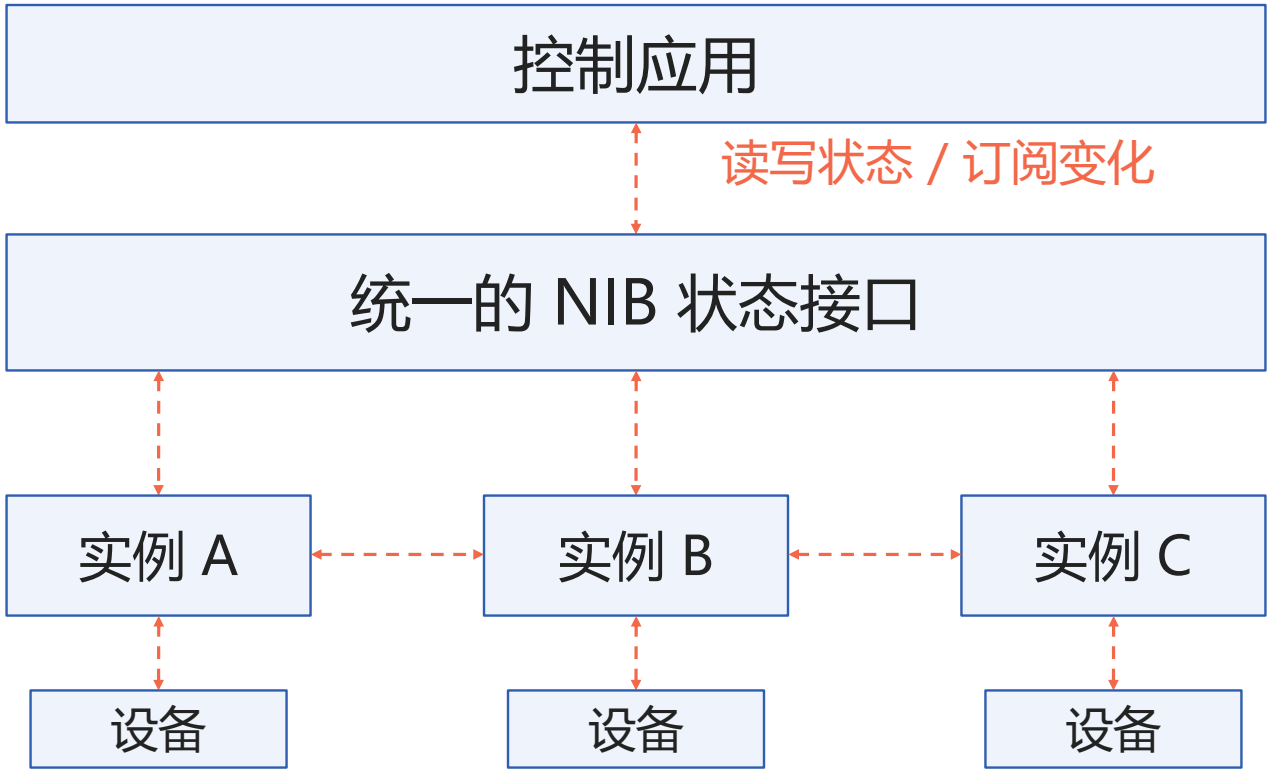
状态共享与职责协调

多实例需要分担状态、事件与计算，并共享状态、检测故障和接管责任

控制器故障后，已安装且仍有效的规则通常仍可执行

从 NOX 到 Onix：把分布式控制做成平台

为支持上述分工与协作，Onix 延续 NOX 的网络编程思路，提供可扩展的分布式控制平台



先明确平台与应用的分工，再看 NIB 接口和多实例协作

应用面向网络状态编程

读取设备、端口和链路等对象，按策略修改所需状态

平台支持多个实例协作

组织状态分发和职责协调，让控制工作能够分担

NIB：Network Information Base

Outline

I. 问题与贡献
II. 架构与 NIB
III. 扩展与可靠性
IV. 状态分发与一致性
V. 实现与应用
VI. 实验评估
VII. 小结

I. 问题与贡献

- 为什么要复用公共控制机制
- 平台与应用分别负责什么
- 生产级要求如何形成两项贡献

为什么要把公共控制机制做成平台

多实例协作需要公共机制；若每种控制功能都重新实现，设计和部署成本就会很高

新增控制功能需要做什么

新增控制功能常需同时实现状态传播、设备发现和故障恢复，并将专用协议部署到交换机。各功能重复实现底层机制，扩展性和收敛行为也各不相同

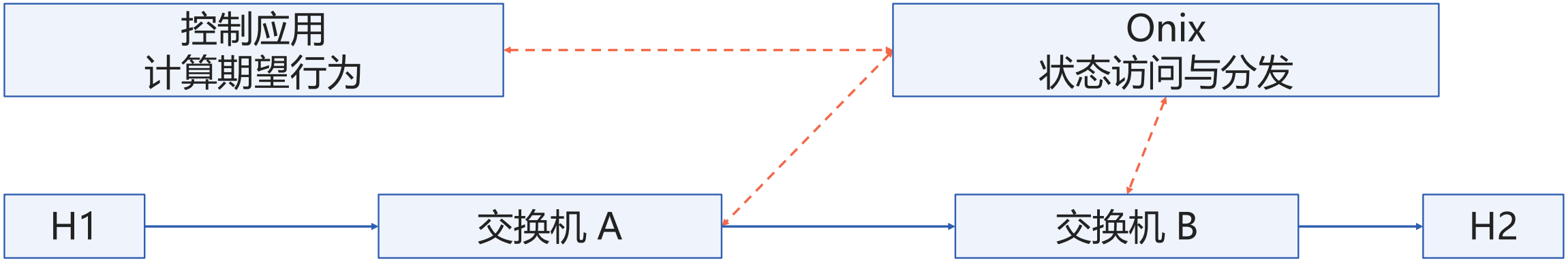
共性工作交给控制平台

Onix 在服务器上复用这些机制，让应用通过高层 API 表达控制逻辑
平台接入设备、分发状态；应用决定需要哪些状态、如何使用

公共机制可以复用，但控制决策与设备的高速转发仍各有分工

Onix 中控制与高速转发的分工

在 Onix 中，应用计算目标状态，平台传递控制更新，物理设备执行高速转发



上层决定网络行为

应用读取状态并计算路径、策略或配置；Onix 提供状态接口，将更新交给设备

下层持续处理报文

论文假设设备转发远快于通用控制平台，因此平台处理必要事件，设备持续处理报文

控制粒度要匹配事件速率、计算成本与响应时限；平台复用的是这些控制操作的公共机制

多种应用共用哪些平台能力

路径控制、访问控制和虚拟机管理的业务逻辑不同，但可复用以下三类平台能力

接入设备状态	平台连接设备、适配协议，将端口、链路和转发状态表示为对象
组织与传播状态	应用读写对象并决定共享范围；平台按配置导入、导出和分发状态
支撑故障与协调	平台支持成员管理、故障检测与协调；应用决定责任转移、路径恢复和冲突处理

论文设想一次部署主要运行一个管理应用，其中可包含多种功能；这些公共能力还需满足生产级要求

生产级平台的五项要求与取舍

要让上述公共能力用于生产网络，平台还需兼顾通用性、扩展性、可靠性、易用性和控制性能

要求	平台要求	对应设计
通用性	支持不同网络环境与控制功能	提供可扩展的网络对象与 API
扩展性	瓶颈来自状态问题本身，而非平台实现	支持应用分区、聚合及分发选择
可靠性	妥善处理设备及其他类型的故障	提供检测、协调与状态恢复支撑
易用性	降低构建管理应用的工作量	复用设备接入和状态管理机制
控制性能	提供足够的控制响应能力	权衡时优先通用性，满足所需性能

控制性能要求“足够”，并非最优；极快恢复需结合设备备份路径等机制。这些取舍形成以下两项贡献

两项贡献：通用接口与分发选择

Onix 用两项设计回应上述要求：通用的网络状态接口，以及可按需求选择的状态分发机制

通过网络状态表达控制

NIB 将设备、端口、链路等表示为可读写对象。应用订阅变化、读取属性并修改对象，平台连接外部网络与数据来源

按应用需求选择分发

状态的更新频率、持久性和冲突代价不同。应用选择共享、分区与协调方式，权衡一致性、持久性和扩展能力

先看应用怎样通过 NIB 完成一次控制，再看多个实例怎样分担和共享这些状态

Outline

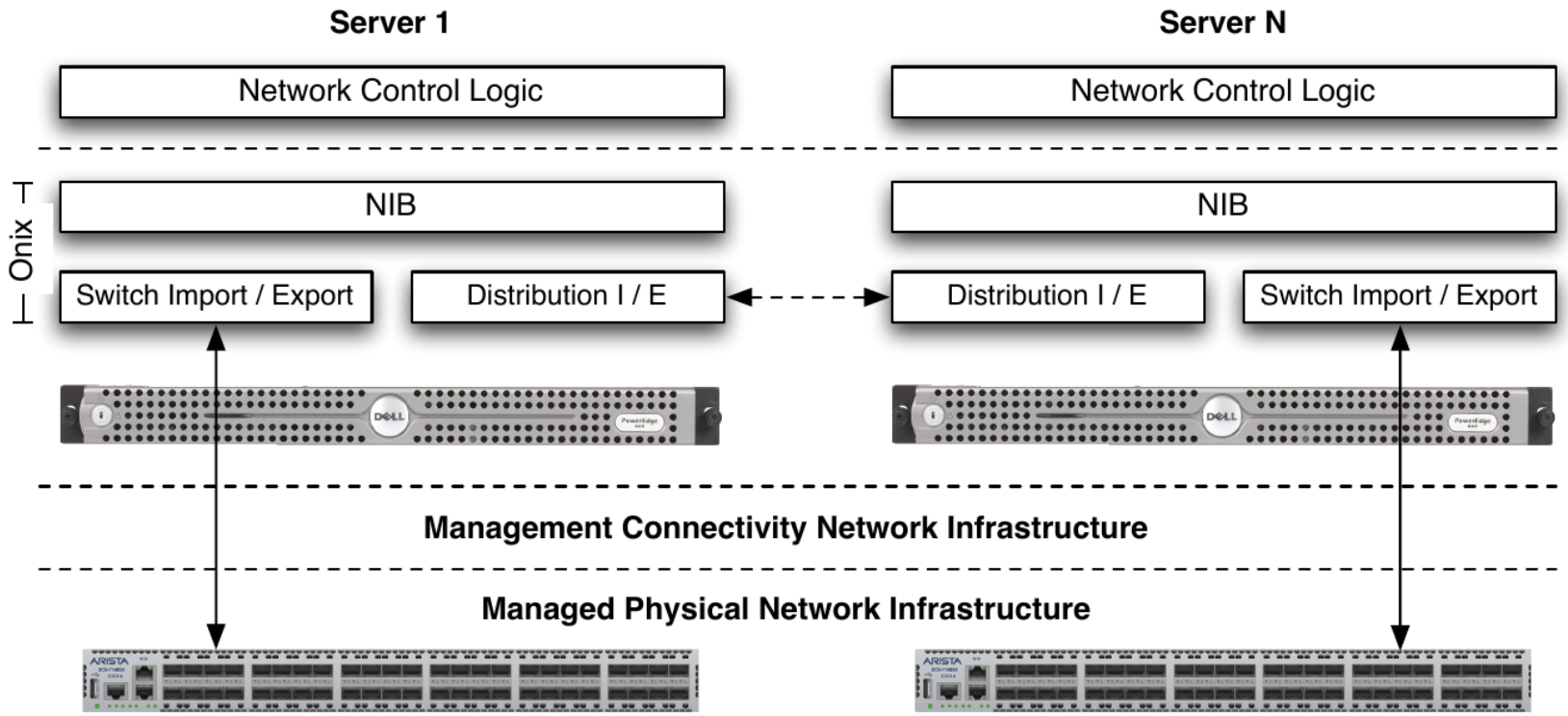
I. 问题与贡献
II. 架构与 NIB
III. 扩展与可靠性
IV. 状态分发与一致性
V. 实现与应用
VI. 实验评估
VII. 小结

II. 架构与 NIB

- 从整体架构看一次设备事件
- NIB 如何表示对象并支持应用操作
- 本地修改、异步导出与同步等待

Onix 网络中的四类组件

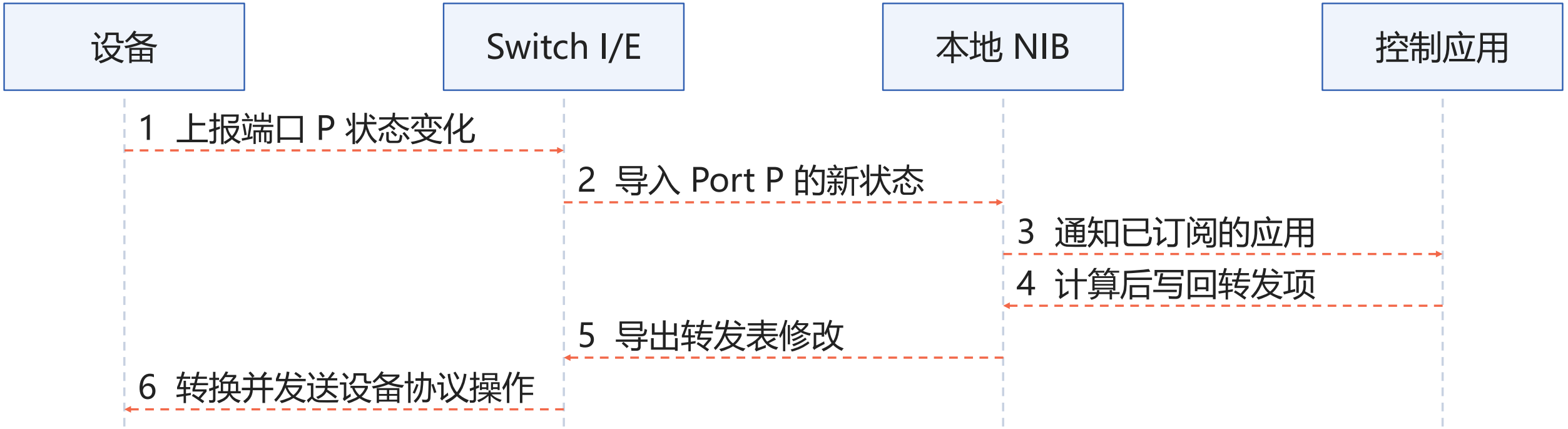
- Physical Infrastructure：交换机、路由器等设备，执行数据转发
- Connectivity Infrastructure：承载设备与实例、实例之间的控制通信
- Onix：通过 NIB 提供状态访问，由 I/E 导入、导出和分发状态
- Control Logic：读取状态并计算路径、流量工程或访问控制策略



来源：Onix Figure 1

一次设备事件如何进入控制逻辑

沿架构图看端口 P 失效：Switch I/E（设备导入/导出）更新 NIB，应用计算后再写回转发状态



NIB 统一的是对象访问接口

上一流程中的设备事件和应用修改，都通过 NIB 对象衔接；应用无需直接处理设备协议



应用看到网络对象

RIB 主要按目的前缀组织路由；NIB 表示节点、端口、链路等对象。应用读写属性、访问引用，I/E 处理协议转换

实例只保留所需状态

各实例只保留需要的状态，可复制、分区或仅存本地。应用配置导入导出范围，并选择一致性与持久性

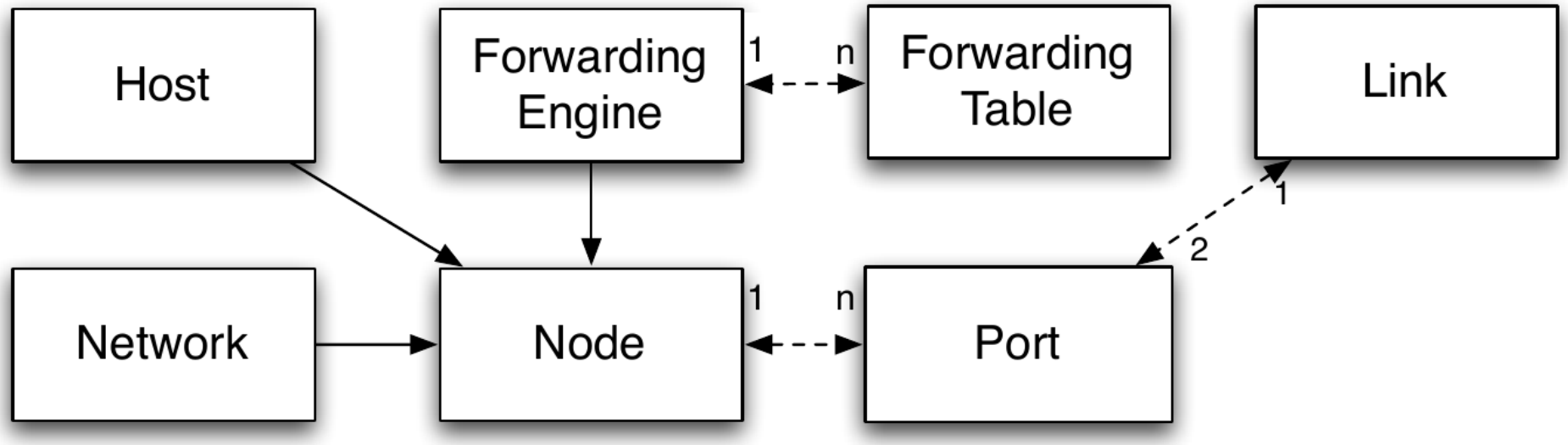
实体、属性与类型各自表达什么

以刚才的 Port P 为例：ID 标识对象，属性保存状态，引用关联对象，类型规定常用操作

组成	解决的问题	以 Port P 举例
实体 ID	用平坦的 128 位全局 ID 标识实体	端口 P 有自己的 ID
键值属性	保存可读写和传播的状态	P 的状态为 UP，流量计数为 100
实体引用	关联相关对象	交换机 S1 包含端口 P
类型与方法	规定常用属性与操作	Port 类型表示端口，应用可继续派生

这四项组成对象模型；下面看 Node、Port 和 Link 怎样通过类型与引用关系组织起来

NIB 对象怎样组成类型与拓扑关系



Onix Figure 2

实线连接类型

Host、Network、ForwardingEngine 继承 Node；所有实体类共享通用键值访问基类

虚线连接实例

Node 关联多个 Port；ForwardingEngine 关联多个 ForwardingTable

基数限定关系

一个 Link 连接两个 Port。应用沿这些实例引用取得拓扑，供事件回调计算路径

应用收到事件后怎样操作 NIB

再看设备事件流程中的应用部分：收到通知后读取相关对象，计算目标，再写回 NIB

应用先注册自己关心的事件



通知、查询和修改分别完成这些步骤；下一页将它们对应到具体 API

NIB API 的七类操作如何配合

上述控制过程对应七类 API：对象访问、变化通知、导出等待和导入导出配置

Category	Purpose
Query	Find entities.
Create, destroy	Create and remove entities.
Access attributes	Inspect and modify entities.
Notifications	Receive updates about changes.
Synchronize	Wait for updates being exported to network elements and controllers.
Configuration	Configure how state is imported to and exported from the NIB.
Pull	Ask for entities to be imported on-demand.

Table 1

查找与修改对象

Query 查找实体；Create / destroy 管理生命周期；Access attributes 读写属性

观察变化与推送

Notifications 通知对象变化；Synchronize 等待推送，回调后由应用核验

导入所需数据

Configuration 配置导入导出；Pull 按需将远端实体导入本地，再通过对象接口访问

例：修改 S1 转发表，需要等待导出时使用 Synchronize

Query 查找 → **Access attributes** 修改 → **Synchronize** 等待 → 应用核验

写入 NIB 后，外部状态异步更新

Access attributes 先修改本地对象，再异步导出；默认不保证外部更新的发送顺序和时延



先写 A 再写 B，不代表外部按此顺序更新。存在先后依赖时，应等待 A 推送、核验后再修改 B

Synchronize 回调与应用核验

为组织有先后依赖的更新，应用对指定实体调用 Synchronize，等待回调后重新读取并核验

01

修改指定实体

将本步期望状态写入指定实体，明确后续操作依赖哪个对象

02

等待状态推送

为该实体调用 Synchronize；状态已推送后，平台回调通知应用

03

重新读取核验

回调后读取 NIB，核验是否符合预期，并考虑期间的并发变化

04

决定下一步操作

符合要求再继续修改；存在差异时，由应用协调或恢复

本地 NIB 独占访问保护什么

同步等待关注外部推送；本地独占访问则保护同一实例内的读写，不会冻结其他实例或设备

访问范围	独占访问能否保护	说明
同一实例内的其他线程	可以	防止其他线程同时修改本地 NIB
其他 Onix 实例	不可以	其他实例仍可能修改相关状态
交换机等网络设备	不可以	设备状态仍可能发生变化

本地独占不等于分布式协调。单实例流程清楚后，还需讨论多实例如何分担状态、事件与计算

Outline

I. 问题与贡献
II. 架构与 NIB
III. 扩展与可靠性
IV. 状态分发与一致性
V. 实现与应用
VI. 实验评估
VII. 小结

III. 扩展与可靠性

- 单实例先遇到哪一种瓶颈
- 分区、聚合分别减少什么负担
- 设备故障与实例故障怎样区分

NIB 的三种扩展瓶颈

NIB 的对象、通知和应用计算都消耗资源。网络扩大时，应先判断哪一种负担超出单机能力

状态容量受限

- 交换机数量不多时，全部转发表项也可能超出单机内存。
- 只保留本实例负责的状态，避免完整复制全部对象

事件更新受限

- 拓扑仍放得下，但频繁的利用率通知可能耗尽 CPU。
- 减少对外传播的细节，避免每个实例处理区域内的每次变化

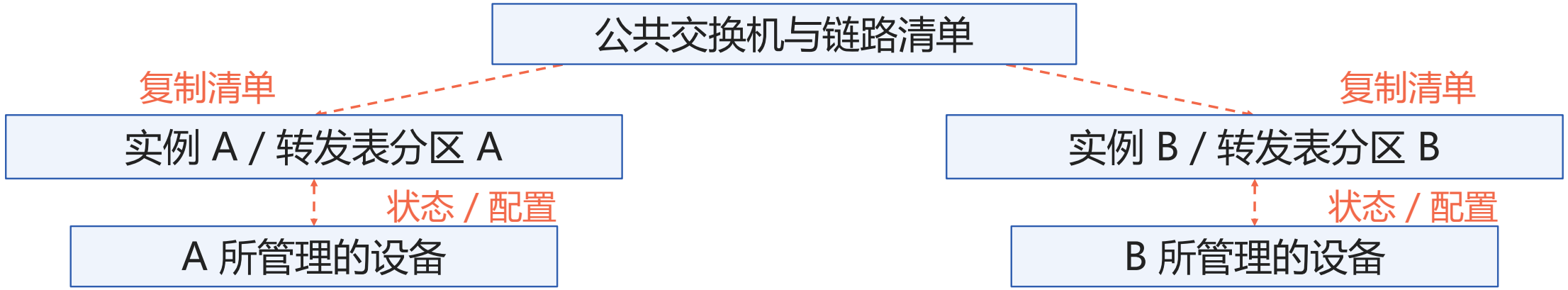
应用计算受限

事件数量相同，算法开销也可能不同。统计或路径重算过重时，需要分担计算职责

增加实例只有减少单实例的实际负担才有效。先看状态过大时如何分区

分区：分担状态与设备管理责任

先处理容量瓶颈：假设完整拓扑放得下、全部转发表放不下，就复制公共清单并分担转发表



共享计算所需的公共信息

各实例仍需要交换机与链路清单来理解完整物理拓扑

减少每个实例的实际负担

应用让每个实例仅维护分给自己的转发状态，并可只连接一部分设备

分区与复制可同时使用；关键是区分哪些状态共同需要、哪些只由负责实例维护

路径应用如何放置三类状态

分区后仍要共享拓扑与利用率。这里先看三类状态的分工，复制库与 DHT 的机制在下一章展开

状态	计算中的作用	放置方式	选择理由
交换机与链路清单	提供端到端路径所需拓扑	由所有实例共享复制数据库	可靠保存，通过协调管理更新
链路利用率	按负载选择合适路径	由内存 DHT 共享，相关实例导入	读数变化快，重视更新效率
转发表状态	将路径决定落实为设备规则	按分区由唯一负责实例管理	分担大量状态与管理工作

同一应用可混用复制库、内存 DHT 与分区管理；若共享更新仍过多，还需聚合

聚合：减少远端需要处理的细节

分区后，共享事件仍可能过多

- 即使完整 NIB 仍能放入内存，频繁的状态变化也可能耗尽 CPU
- 方式一：聚合链路利用率
  - 保留完整拓扑，汇总区域内的利用率后共享，减少远端处理的更新
- 方式二：聚合网络拓扑
  - 在层次化部署中，将区域抽象为逻辑节点，供上层计算跨区域路径
- 核心思想：减少远端接收的细粒度信息，降低事件处理开销

区域内处理细节 → 对外暴露所需信息 → 远端据此决策

分区减少各实例负责的状态，聚合减少远端观察的细节；保留多少信息取决于决策需求

瓶颈不同，扩展办法也不同

分区与聚合可以组合。控制域继续扩大时，还可细分管理范围、组织跨区请求，并限制暴露的信息

变化的约束	应用采用的办法	仍需保留的工作
转发状态超过单机内存	把转发表分给唯一负责实例	复制公共拓扑以计算端到端路径
高频事件耗尽处理能力	降低远端收到的利用率细节	本区域处理原始变化并形成摘要
实体规模继续扩大	细分管理范围，以属性传递跨区请求	通过 DHT 交换请求与响应状态
跨自治域需保护隐私	暴露适当拓扑摘要及入口信息	按约定或按需为对方建立路径

这些方法回答“工作怎样分担”；责任分开后，还必须考虑设备、实例或控制连接失效时怎么办

可靠性：先区分故障发生的位置

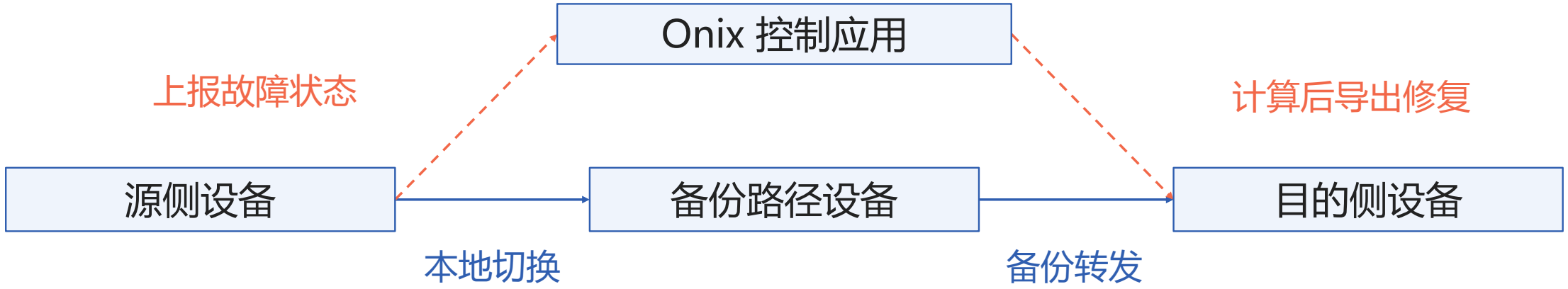
分区解决工作分担，可靠性还需区分四类故障：它们影响的业务路径、管理责任和控制通道不同

故障对象	受影响的工作	响应方式
转发设备	设备上的路径和配置无法继续使用	传播故障状态，计算绕行并修复转发
网络链路	故障链路上的业务路径受影响	可先用预置备份路径保护，再重算修复
Onix 实例	负责的设备管理与应用计算中断	借助协调设施检测故障，调整管理责任
控制连接	设备访问或实例间分发中断	恢复底层可达性，为控制修复提供通道

检测后仍需传播状态并执行修复。先看设备与链路故障的快速保护，再看多实例共同控制的问题

本地快速保护与控制器修复

对设备或链路故障，快速保护可先在设备侧完成；控制器收到故障状态后再重算和修复



设备先保护受影响的流量

设备已配置备份路径和切换机制时，可在检测到可处理的故障后先保护流量，无须等待控制器重算

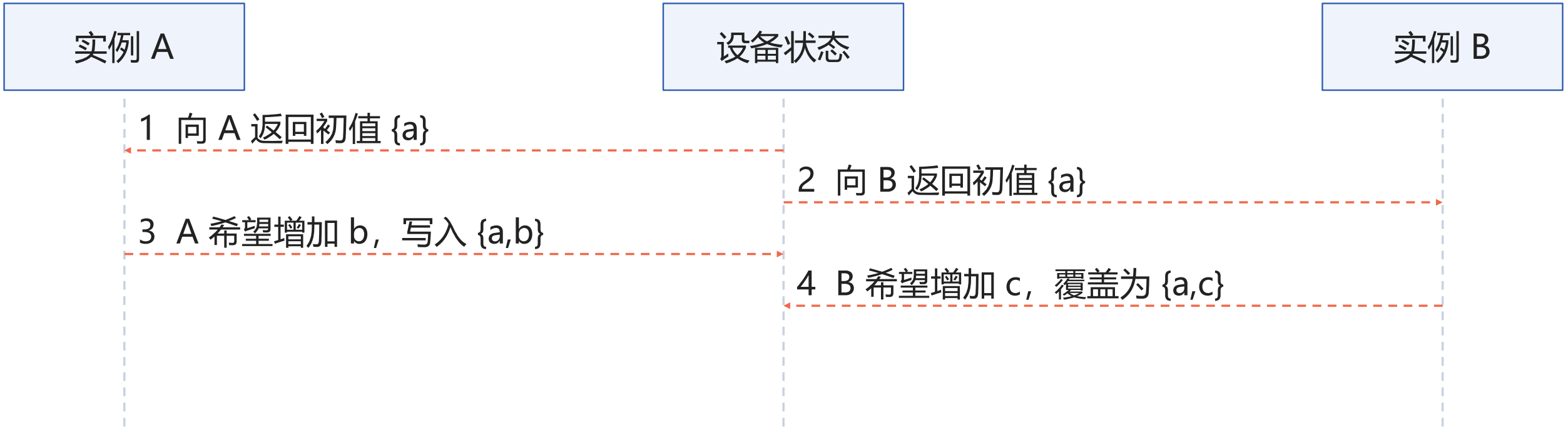
应用完成后续网络修复

应用收到故障状态后重算转发表并导出修改。恢复速度受传播、计算和控制连通性共同约束

本地保护与控制修复可以配合；具体保护范围和时限由部署决定。多个控制实例还需协调彼此的更新

主动—主动控制中的更新覆盖

为保持控制服务，可由多个实例同时管理设备；但若都根据旧值整体写入，就可能相互覆盖



本例两次操作都要保留已有元素，但设备接受整集合替换，因此 B 的旧值覆盖丢失了 b。Onix 提供钩子，由应用决定能否覆盖冲突修改。相同算法确定性计算同一目标时，不一致可只暂时存在，处理语义仍由应用定义

Outline

I. 问题与贡献
II. 架构与 NIB
III. 扩展与可靠性
IV. 状态分发与一致性
V. 实现与应用
VI. 实验评估
VII. 小结

IV. 状态分发与一致性

- 共享状态应选择哪一种后端
- I/E 怎样衔接状态与设备协议
- 不一致与职责交接由谁处理

状态性质决定分发与存储选择

前面已看到共享与冲突问题。选择分发机制时，要看状态变化多快、能否重建，以及冲突后果

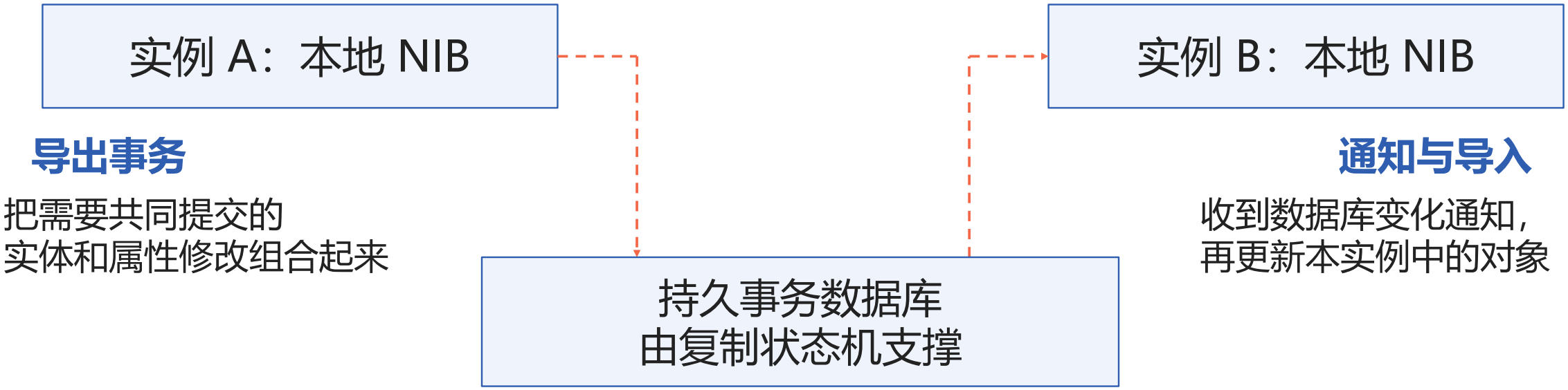
状态	更新与故障后果	设计指导
策略声明	慢变；丢失或冲突影响全网策略	持久保存并加强协调
链路负载	持续更新；新读数替代旧读数	更新率优先，可用内存 DHT 分发
链路状态	相邻链路标志可能暂时不一致，可重建	应用处理暂态，可放宽同步要求
端点位置	共享范围可能是全网或资源池	按读取需求选择复制、分区或本地保存

同一应用可按实体或属性混用后端。先看适合持久配置的复制库，再看服务高频更新的 DHT

方式一：复制数据库保存慢变状态

策略、配置和可靠清单适合复制数据库；复制状态机支撑持久事务，I/E 连接数据库与本地 NIB

以实例 A 发起一次修改为例

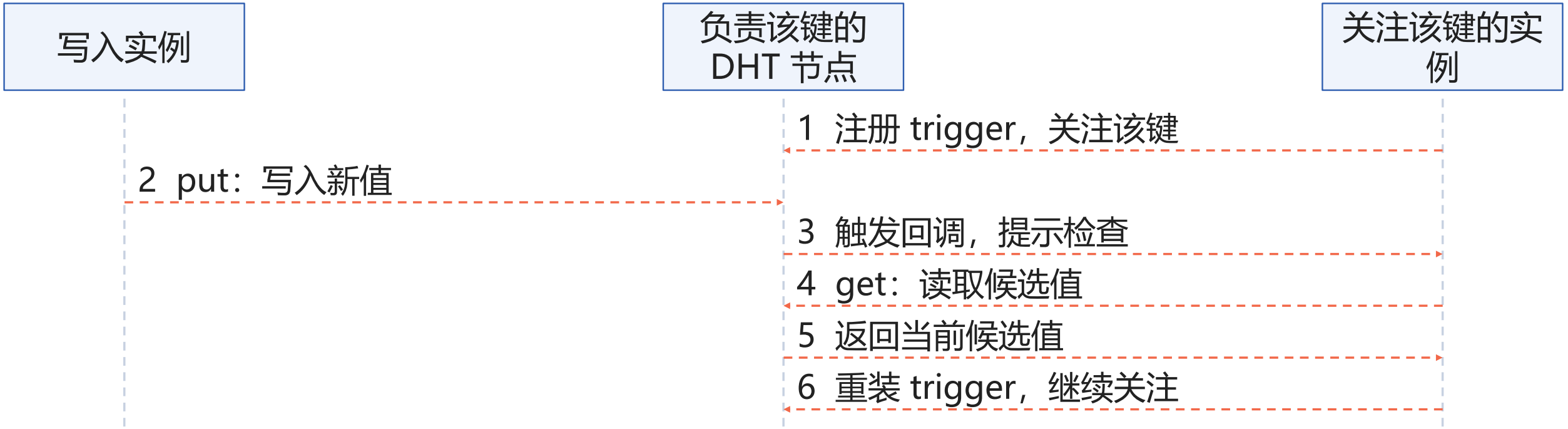


共同提交的是数据库修改；各实例再把变化导入本地 NIB

远端 NIB 导入和设备操作仍各自推进，并非跨设备原子更新。复制事务代价较高，快变状态另用 DHT

方式二：DHT 分发高频更新

链路负载等状态更重视更新率。Onix 提供一跳、仅内存、最终一致的 DHT，通过软状态触发器提示读取



- trigger 触发后需重新注册
- 通知可能误触发，收到通知后仍需读取确认
- 并发写入可能返回多个冲突值，由应用解决

复制数据库与 DHT：选择理由及代价

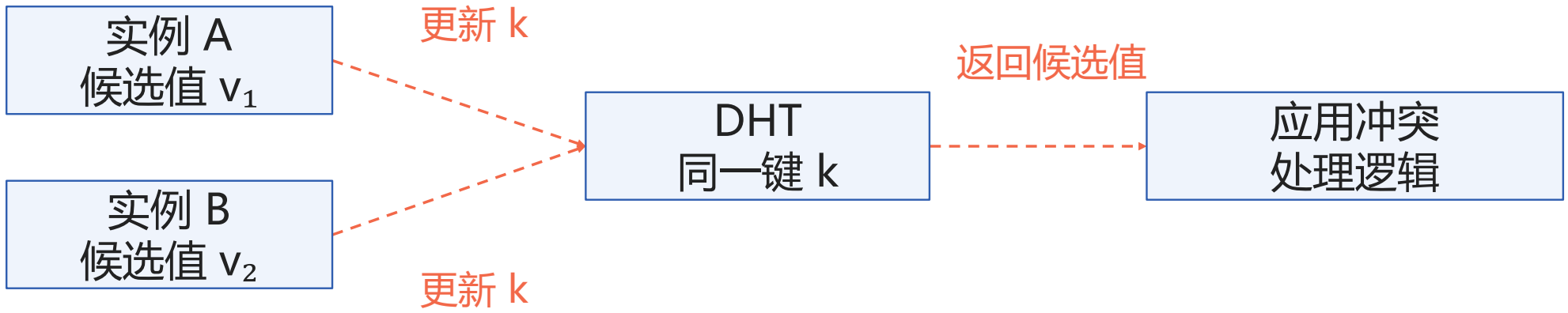
两种后端并非互相替代：复制库偏重持久性与事务，DHT 偏重高更新率与可用性

比较维度	复制事务数据库	一跳内存 DHT
适合的状态	慢变策略、配置和可靠清单	快变、可重新观测的链路负载
应用访问	SQL 查询与 trigger；I/E 可成组导出事务	get、put 与软状态 trigger；触发后重装
提供的语义	事务与复制状态机支撑持久化及一致性	仅内存、最终一致；并发时可能返回多个值
代价	复制事务限制吞吐，宜控制更新率	应用消解候选值，或协调避免双写

路径应用可把清单放复制库、利用率交给 DHT。选择 DHT 时，必须理解多个候选值如何处理

DHT 冲突：同一键出现多个候选值

展开 DHT 的并发写入问题：责任交接时，新旧负责者可能短暂同时更新同一个计数键



多个候选值需要应用判断

例如 A 写入 k = 100，B 写入 k = 120；并发更新时，读取可能得到两个候选值

处理结果要同时更新本地与共享状态

应用按状态含义选择或合并候选值，再更新本地 NIB，并把结果写回 DHT

I/E 配置决定状态的来源、去向与范围

无论读取候选值还是写回结果，都经 I/E 衔接。应用配置状态的来源、去向和共享范围

导入：把外部状态变成对象

从数据库、DHT 或设备取得实体与属性，写入本地 NIB。应用指定所需数据及来源，也可按需导入远端实体

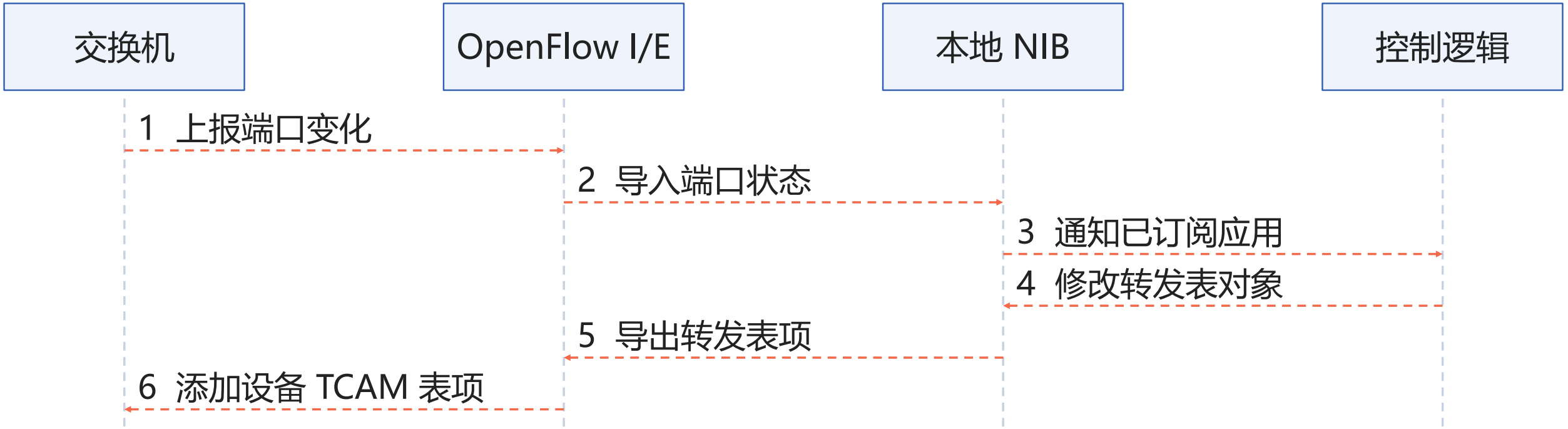
导出：把对象变化交给外部机制

把 NIB 修改交给选定存储或设备。不同状态可用不同出口，应用也能编写自定义 I/E 接入其他存储

部分状态可只存本地；责任调整时修改 I/E 范围。下面分别看流表更新和设备配置如何导出

设备 I/E：把 NIB 修改转成 OpenFlow 操作

回到端口故障例子，这次看协议转换：OpenFlow I/E 将端口事件导入 NIB，再把转发表修改导出

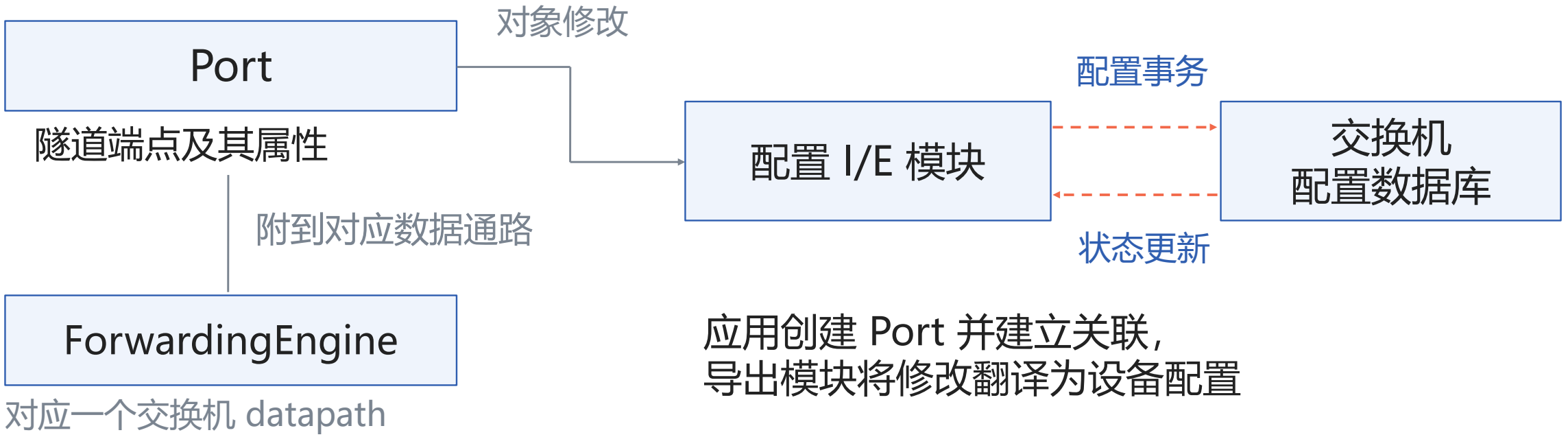


应用修改 ForwardingTable，I/E 翻译成设备流表操作；论文以 TCAM 表项为例。OpenFlow 之外的配置，还需相应的配置协议

创建隧道端点：从 Port 对象到设备配置

隧道端点等一般配置不都由 OpenFlow 暴露；应用创建 Port 并建立关联，由配置 I/E 写入设备

NIB 中的对象



这里访问设备配置数据库，并非实例间共享状态的复制库；其查询语言更受限，不提供 join。多种来源汇入 NIB 后，还要处理到达次序

跨来源不一致：Node 已到，Port 未到

不同来源没有统一到达次序：复制库的 Node N 可先引用设备端口 P，而 Port P 对象稍后才导入



问题：引用对象尚未到达

N 已导入而 P 尚未到达；直接暴露 N 对 P 的引用，会出现引用不一致

解决方式：暂时隐藏引用

应用扩展实体类，在 P 导入前暂不暴露引用，待 P 到达后再提供关联访问

这是对象引用缺失，不同于 DHT 同键多值；下一页归纳各类问题的处理位置

三类状态不一致及处理方式

将前面的例子放在一起：跨来源引用缺失、同源候选值和本地/外部双写，需要分别处理

出现在哪里	具体状态	应用如何处理
跨来源的引用	Node 已到，被引用的 Port 未到	实体子类暂缓暴露引用
同一数据源内部	职责重叠，同一 DHT 键出现多值	应用消解后更新 NIB，并写回 DHT
本地与外部之间	本地与外部同时修改同一状态	I/E 协调导入值与本地修改

实体类检查引用，I/E 协调值冲突；分布式协调可减少双写，也可支撑故障后的责任接管

ZooKeeper 支持协调，应用调整控制责任

回到实例失效与责任交接：Onix 封装 ZooKeeper 层次命名空间接口，应用据此组织协调和接管

平台提供协调工具

ZooKeeper 接口

提供层次命名空间的对象接口

成员与故障信息

为组成员管理、故障检测提供支持

协调算法的基础

应用可据此组织共识等协调算法

应用决定怎样接管

重新划分责任

决定哪些状态和设备由谁接手

调整导入与导出

按新的责任范围读取状态、输出修改

继续控制并处理冲突

执行应用逻辑，处理交接期的并发更新

ZooKeeper 提供协调基础，不自动决定业务责任。多数持久配置仍放复制库：ZooKeeper 对象大小受限，NIB 访问也更方便

Outline

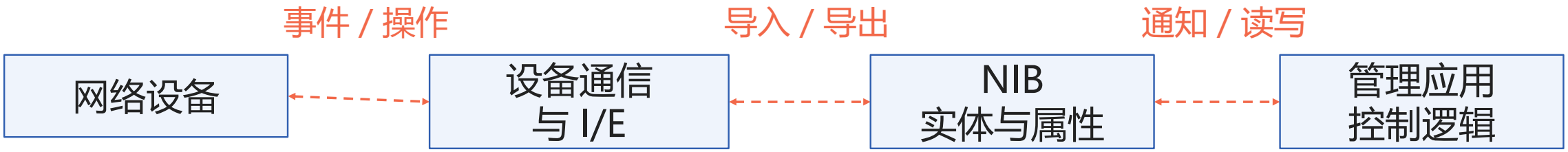
I. 问题与贡献
II. 架构与 NIB
III. 扩展与可靠性
IV. 状态分发与一致性
V. 实现与应用
VI. 实验评估
VII. 小结

V. 实现与应用

- 平台如何实现前述接口与机制
- Ethane 与 DVS 的控制需求为何不同
- 多租户如何分区并共享隧道端点

Onix 的实现：设备通信、NIB 与应用框架

上述对象接口、分发和协调机制组成控制平台；论文实现约十五万行 C++，并集成第三方库



基础框架负责共性的底层工作

- 设备通信将观测转换为 NIB 状态，并提供访问和通知
- 应用计算目标，修改经适配模块返回设备，复用底层机制

通用能力通过组合实现

- 复制库、DHT 与协调设施共同支撑状态管理；实例内外通信复用 RPC
- 组件在接口不变时可替换。下面看不同应用怎样组合这些能力

应用总览：不同需求使用不同平台能力

同一平台支持不同应用。表中勾选表示特别强调的能力，下面重点比较控制触发、状态共享和故障影响

Control Logic	Flow Setup	Distribution	Availability	Integration
Ethane	✓		✓	
Distributed virtual switch				✓
Multi-tenant virtualized datacenter		✓		✓
Scale-out carrier-grade IP router			✓	

来源：Table 2

新流建立与可用性

Ethane 每条新流都要作策略判断，因此重视首包处理能力和控制服务连续性

虚拟化与状态分发

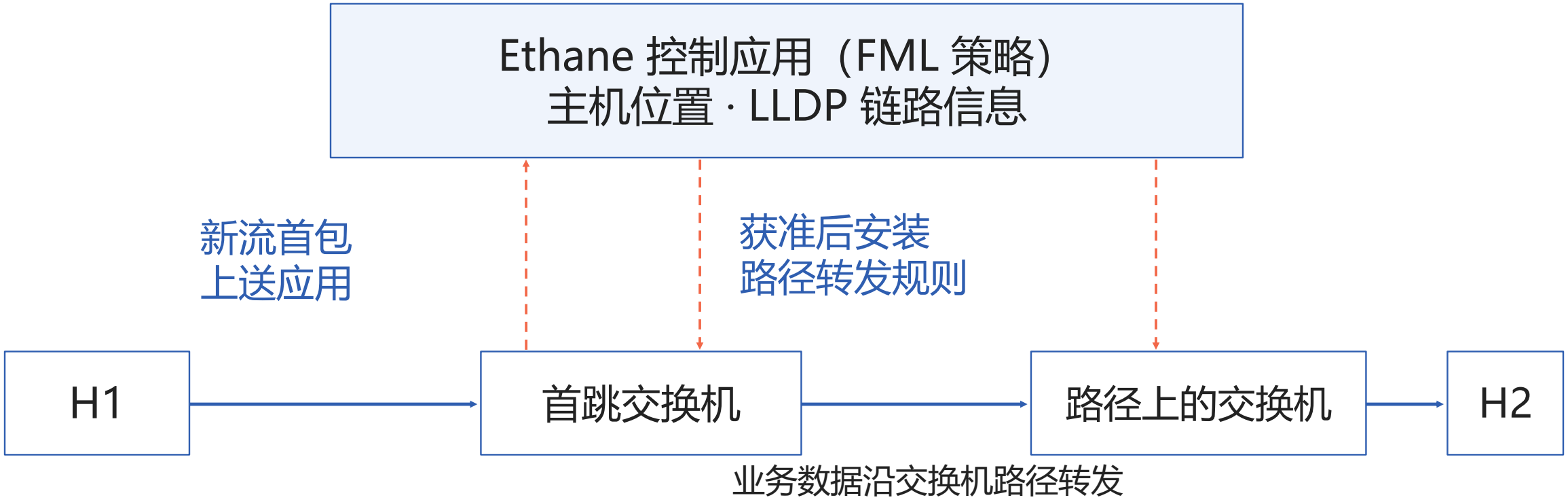
DVS 跟随 VM 生命周期；多租户还要分担隧道端点状态，分别突出集成与分发

路由器方案仍在设计阶段

逻辑路由器方案强调可用性，但论文发表时仍处设计阶段，并非已完成部署验证

Ethane：新流首包触发策略判断与路径建立

先看按新流触发的 Ethane：用 FML 表达策略，结合主机位置和链路状态，为获准流建立路径



控制器参与新流建立，失效会阻碍新通信。因此规模扩大时，需分担新流处理并保留必要网络视图

Ethane：分担新流处理，保留端到端视图

新流总量超出单机能力时，处理工作可以分担
但负责某条流的实例仍需全网端点与链路状态，才能计算完整路径

对象或条件	怎样组织	为什么
新流处理工作	多个实例各承担一部分流	减少单机工作，不重复处理全部流
端点与链路视图	各实例保留全网端点与链路状态	负责实例仍需计算完整路径
链路状态存储	经 DHT 分发，可动态发现和重建	放宽持久性与同步要求以支持变化
暂态与故障	过时位置或链路可能导致临时错误路径	处理暂态，实例故障后恢复新流处理

DVS：按资源池管理，冷备保护控制操作

与 Ethane 每条新流触发不同，DVS 主要在 VM 创建、销毁和迁移时控制，使端口策略随 VM 移动

资源池给出了自然的管理边界

Hypervisor 按资源池组织，VM 通常不跨池迁移。每实例管理一个池；配置持久保存到事务库，VM 位置无需跨实例共享

控制失效首先阻碍 VM 动态变化

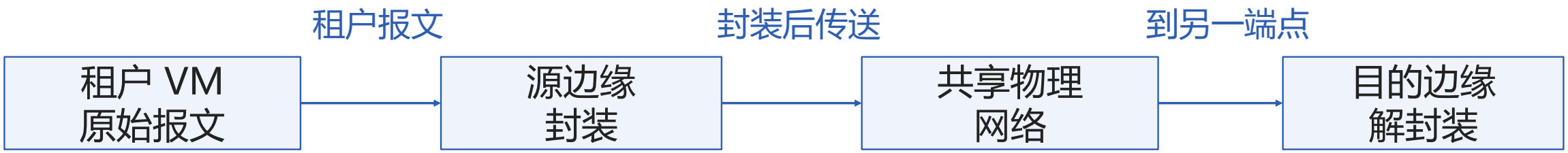
实例失效时，已有网络仍运行，VM 动态操作暂停。因即时切换要求低于 Ethane，采用故障后启动的冷备

VM 生命周期事件 → 更新逻辑端口与策略关联 → 配置相应虚拟交换机

DVS 以资源池限制共享范围；多租户网络则还要管理跨 Hypervisor 的隧道关系

多租户网络：在边缘封装，提供独立二层网络

多租户不仅要跟随 VM 变化，还要隔离地址和资源：租户可使用重叠地址，独立配置二层网络



在进入物理网络前建立隔离

源 Hypervisor 封装租户报文，经物理网络传送；目的 Hypervisor 解封装后交付目标 VM

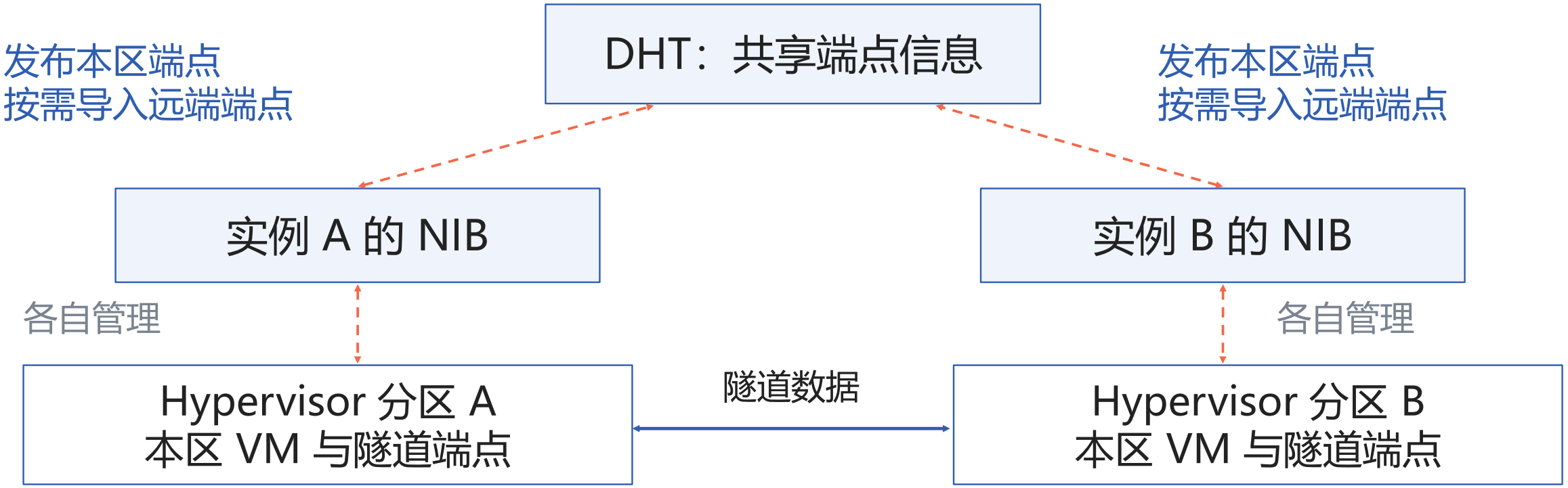
独立服务带来需要管理的隧道关系

应用在同租户涉及的 Hypervisor 间建立成对隧道；关系可能达 $O(N^2)$，增加状态管理负担

因此，隧道端点不能都交给一个实例：需要分区管理，并共享建隧道所需的信息

隧道端点：分区管理，并在建隧道时导入

针对平方级隧道关系，实例按 Hypervisor 分区管理本地端点，并通过 DHT 按需导入远端端点



分区和按需导入降低单实例的状态与连接负担，但未消除全网 $O(N^2)$ 隧道关系，必要端点配置仍需维护

Outline

I. 问题与贡献
II. 架构与 NIB
III. 扩展与可靠性
IV. 状态分发与一致性
V. 实现与应用
VI. 实验评估
VII. 小结

VI. 实验评估

- 本地对象与设备接入有多大开销
- 共享吞吐和传播时延怎样权衡
- 完整应用故障后中断多久

实验分别验证哪些平台能力

测量层级	实际执行的工作	结果
本地 NIB	修改整数属性并通知空监听器；测 NIB 进程内存	Fig. 3–4：修改 ops/s、进程内存
设备协议	接收 64 B 包并返回单包转发指示，不更新流表	Fig. 5：不同连接数下的包/s
实例间共享	RPC 调用、五节点 DHT 传播、复制库混合查询	Fig. 6–7、Table 3：调用/s、ms、查询/s
完整应用	交换机失效后重建隧道、更新转发表	Fig. 8：含检测、决策和修复的总中断

NIB 监听开销与修改批量

先测本地 NIB：每轮独占访问中修改整数属性、通知监听器，再释放访问权；回调不执行业务计算

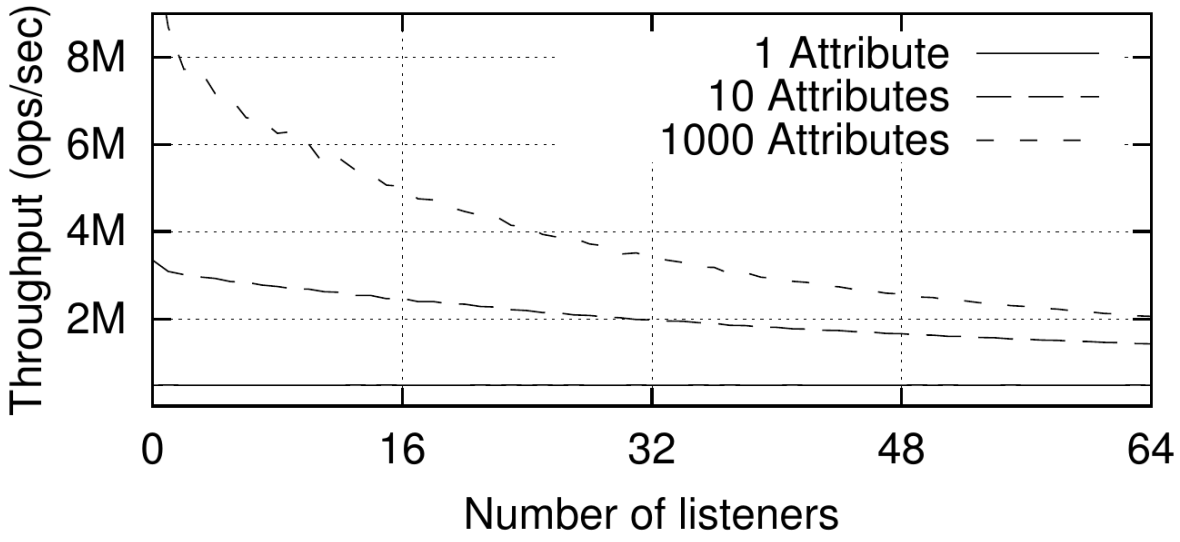


Figure 3

横轴与三条曲线

横轴为监听器数，纵轴为修改次数/秒；三条曲线对应每批 1、10、1000 个属性

批量为何影响吞吐

独占访问伴随上下文切换；批量修改可摊薄每次操作的切换成本

监听数量的影响

监听器越多，通知越多；多属性批次的吞吐随之下降

每批只改一项时，线程切换影响较大；真实回调若计算路径，还会增加开销。下一页看本地状态的内存成本

实体属性与 NIB 内存成本

本地性能还包括保存能力：改变实体数和属性数，测量承载 NIB 的整个 C++ 进程内存

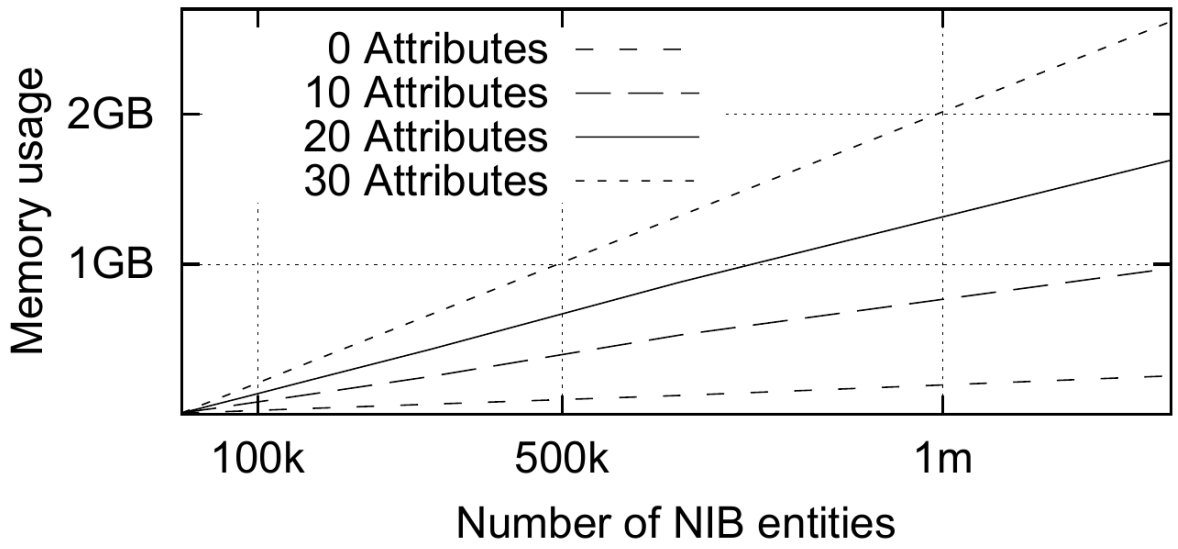


Figure 4

读轴与配置

横轴为实体数，纵轴为进程内存；每实体分别配置 0、10、20、30 个属性

一个实体占什么

零属性实体含存储和索引占 191 B。每属性平均 16 B，另有 8 B 标识及字符串开销

趋势如何解释

内存随实体数近似线性增长，属性越多斜率越大，另有容器管理开销

作者据给定表示推断，服务器级实例可容纳百万级实体；放得下不代表处理得过来，还需测设备事件与通信

OpenFlow 连接与单包处理能力

再测设备接入：单实例接收软件交换机上送的 64 B 包并返回转发指示，全程不更新流表

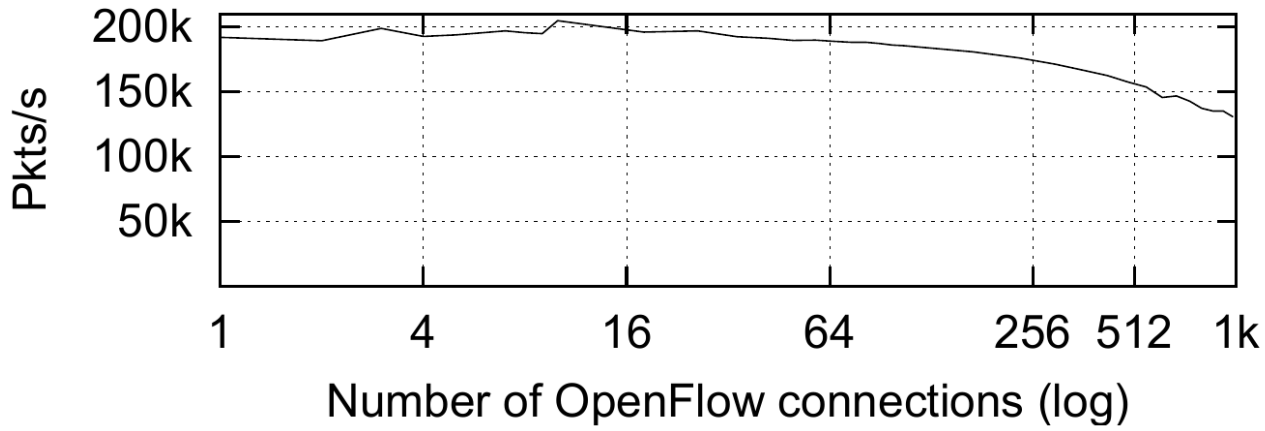


Figure 5

测量协议栈

横轴为连接数（对数），纵轴为包/s；简单应用主要反映 OpenFlow 栈的开销

趋势与原因

约千连接时仍超十万包/s；连接增多带来线程切换，局部起伏来自多核调度

对事件处理的启示

包与端口事件使用相同代码路径；真实计算仍有额外开销。多实例协作还需承担实例间通信

RPC 吞吐与 DHT 更新开销

先测一对实例间的流水 RPC。DHT 每次更新还包含写入、通知与读取等多次调用，需再估算更新能力

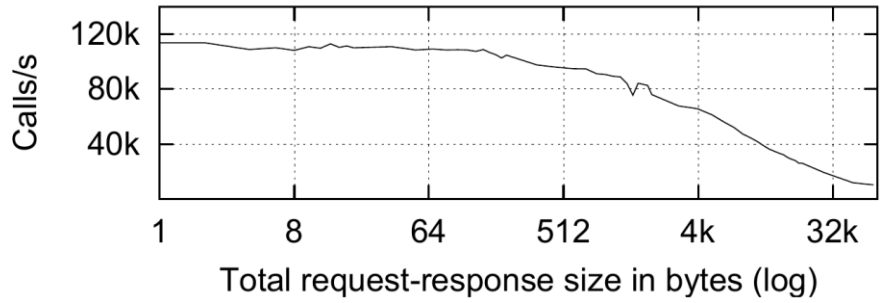


Figure 6

RPC 图的量纲

横轴为请求与响应总字节数（对数），纵轴为调用/s；消息越大，调用速率越低

一次更新共 22 对

五实例全复制 NIB，DHT 两副本：写 2 对、通知 2×5 对、读取并重装触发器 2×5 对

作者的容量估算

结合小消息 RPC 吞吐，作者估算五实例合计约 24,000 次小值更新/秒

2 + 10 + 10 = 22 对 RPC；估算依赖小值、两副本与五实例全复制，并非直接实测 DHT 吞吐。还需看传播时延

DHT 更新何时传播到其他实例

吞吐之外还要看状态何时可用。五节点局域网中，DHT 通知提示读取，关注实例再取回新值

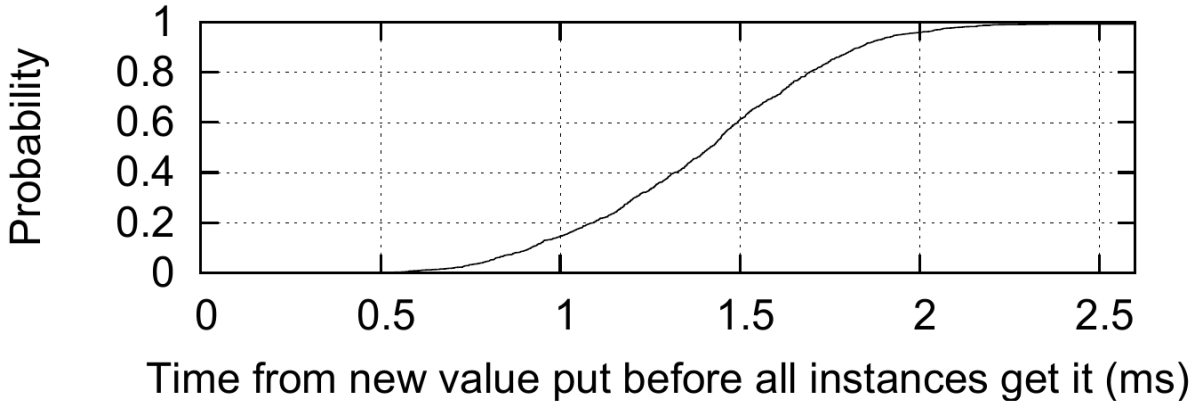


Figure 7

先按 CDF 读图

横轴为写入后时延（ms），纵轴为累计概率；曲线主体约 1–2 ms，仅为读图量级

一次传播的路径

put 写入 → 通知关注者 → get 请求 → 返回值，共四段单向传递

距离增大后的代价

存储节点可能偏离直达路径；论文讨论的最坏情形可比直接推送慢到四倍

批处理与复制数据库吞吐

再看慢变状态的复制库：五节点执行混合查询，1/3 为 INSERT、2/3 为 SELECT，改变每事务查询数

Queries/trans	1	10	20	50	100
Queries/s	49.7	331.9	520.1	541.7	494.4

Table 3

原表中的吞吐

单位是查询数/秒，不是事务数/秒

如何解释批量收益

批量分摊事务固定成本；1 到 20 项提升明显，50 到 100 项未继续增加吞吐

状态选择的依据

持久化与事务有性能代价，因此适合端口清单、策略等慢变状态，而非高频负载读数

无批处理时，原文举例约 17 次端口增删/秒，另有每节点约 6.5 次存在性查询/秒。批量收益并非无限增长

交换机失效后的通信中断

最后测完整应用：隧道承载交换机失效后，重选交换机、创建端点并更新转发表；每 10 ms 用 ping 观测中断

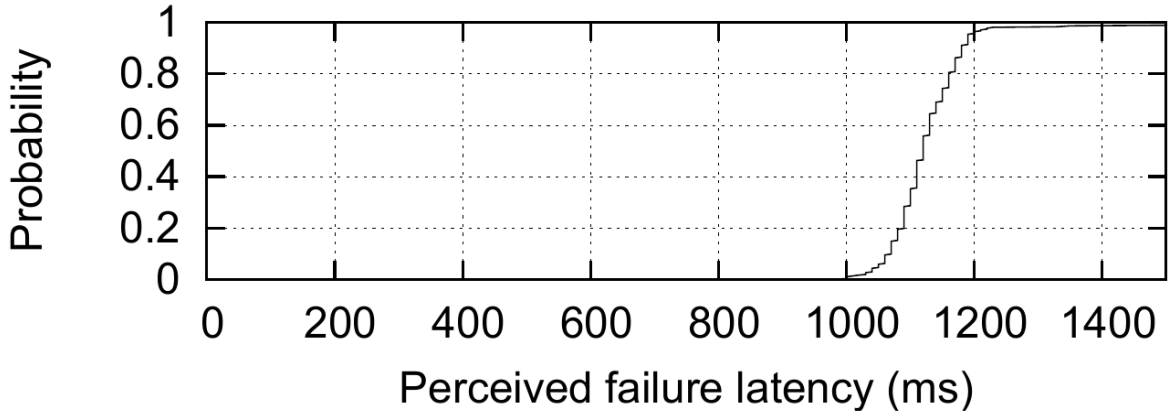


Figure 8

主机测到的总中断时间

横轴为中断时间（ms），纵轴为累计概率；中断包含检测、决策与隧道修复

中位数含检测配置

中位中断 1120 ms，包含应用配置的 1 s 保活超时

约 120 ms 的含义

1120 ms 减去 1 s 后约为 120 ms，是作者推断，并非独立阶段计时

这是交换机失效，不是控制器失效；尚未优化的结果说明检测配置与修复工作共同决定中断时间

Outline

I. 问题与贡献
II. 架构与 NIB
III. 扩展与可靠性
IV. 状态分发与一致性
V. 实现与应用
VI. 实验评估
VII. 小结

VII. 小结

小结：平台能力与应用责任

回顾设计与实验：Onix 提供统一状态接口和分布式机制，应用仍负责把它们组织成正确的控制过程

平台把低层工作变为状态接口

- NIB 支持对象读写和变化订阅
- I/E 对接设备协议、复制库与 DHT
- ZooKeeper 支持成员管理、故障检测与协调

应用决定状态含义与处理规则

- 按状态选择复制、分区与聚合
- 对象缺失时检查引用，条件满足后再暴露
- 责任交接时处理冲突，并调整 I/E 范围

应能区分“平台提供什么”和“应用还要做什么”；下一页将这些责任归纳为可按场景选择的设计

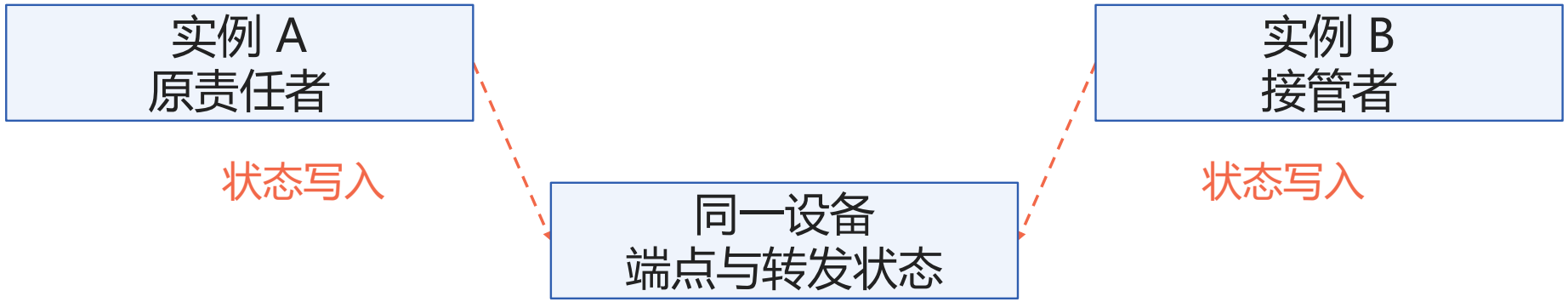
小结：Onix 的设计原则

按对象接口、状态性质、单机瓶颈和恢复时限选择机制；下一页用多租户情景检验这些选择

设计选择	适用情形	在 Onix 中的做法
对象接口	屏蔽设备协议细节	修改 NIB 对象， 由 I/E 转成设备操作
按状态选后端	持久配置与频繁更新 有不同要求	分别使用复制数据库、DHT； 应用负责必要的冲突处理
按瓶颈分工	状态或远端事件过多	分区管理责任，聚合信息， 减少各实例处理的细节
按时限恢复	通信对中断时间敏感	预设备份路径保护数据面， 再由控制应用修复网络

综合讨论：多租户状态与故障接管

教学情景：端点随迁移频繁变化，访问策略低频更新且须持久保存。责任从 A 交给 B 时，两者可能短暂同时写设备



两类状态怎样共享

策略、端点信息与本地状态分别怎样保存？说明复制库、DHT 或本地保存的理由，以及共享范围对可用性和更新开销的影响

职责交接怎样完成

按检测故障、确定新责任者、调整 I/E、恢复设备状态说明接管过程；再分别讨论旧实例覆盖和 DHT 同键多值如何处理